

< 연습문제 #01 > (신경망 기초)

1. 아래 코드의 결과를 정리하시오.

```
import numpy as np
y = np.array([[0.1, 0.2, 0.7], [0.2, 0.8, 0.1]])
print(y)
print(np.argmax(y, axis=0))
print(np.argmax(y, axis=1))
```

y	np.argmax(y, axis=0)	np.argmax(y, axis=1)
<code>[[0.1 0.2 0.7] [0.2 0.8 0.1]]</code>	<code>[1 1 0]</code>	<code>[2 1]</code>

2. 784개의 뉴런으로 구성된 입력에 대해 3층 신경망을 통과한 후 10개의 뉴런으로 구성된 출력을 연산하는 신경망 가중치의 모양을 구하시오(단, 1층 뉴런에 대한 가중치 W2는 90x60) (편향 무시).

Layer	Input	1 st Layer (Hidden1)	2 nd Layer (Hidden2)	3 rd Layer (Output)
Nueron	784	90	60	10
Weight	-	W1 = 784 x 90	W2 = 90 x 60	W3 = 60 x 10

3. 가중치 $W = (1, 2, 3)$, 기울기 $G = (-10, 150, 20)$, 학습을 $\eta = 0.1$ 일 때,

$$W = W - \eta \cdot G$$

(1) 다음 단계의 W를 경사법으로 구하시오.

`W[0] = 1 - 0.1 * (-10) = 1 + 1 = 2`
`W[1] = 2 - 0.1 * 150 = 2 - 15 = -13`
`W[2] = 3 - 0.1 * 20 = 3 - 2 = 1`

`W = (2, -13, 1)`

(2) 변화율이 가장 큰 매개변수의 원소는?

`|ΔW[0]| = 0.1 * |-10| = 1`
`|ΔW[1]| = 0.1 * |150| = 15`
`|ΔW[2]| = 0.1 * |20| = 2`

변화율이 가장 큰 원소는 `W[1]` 이다.

(3) `W[0]`의 경우 경사법을 적용하면 감소하는가 증가하는가?

`W[0]`의 기울기는 음수이므로 증가한다.

(4) W[1], W[2]의 경우 경사법 적용에 의해 감소하는 정도가 크게 차이가 나는 이유는?

학습률 η 가 동일하므로 감소량은 기울기 크기에만 비례한다.

W[1]의 기울기가 W[2]보다 7.5배 크므로 감소량도 15와 2로 7.5배 차이가 난다.

4. 입력 데이터 개수(N) = 1, 원핫인코딩 형식의 레이블 $t = (0, 0, 0, 1)$ 일 때,

$$\text{Loss} = \text{교차엔트로피오차(CEE)} = \text{np.sum}\{-t * \text{np.log}(y)\} / N$$

(1) 아래와 같은 추론치 y_1, y_2 에 대한 CEE를 계산하시오.

y_1 에 대한 CEE = $-\log(0.5) = 0.69$

y_2 에 대한 CEE = $-\log(0.7) = 0.36$

(2) y_1 에 대한 손실값보다 y_2 에 대한 손실값이 줄어든 이유는 무엇인가?

y_2 는 정답 클래스에 y_1 보다 높은 확률을 부여했기 때문이다.

CEE = $-\log(\text{정답 확률})$ 이므로 정답 확률이 클수록 손실 값은 작아진다.

```
import numpy as np

y1 = np.array([0.2, 0.1, 0.2, 0.5])
y2 = np.array([0.1, 0.1, 0.1, 0.7])
t = np.array([0, 0, 0, 1])

# 출력 결과를 print() 옆에 정리
print(np.log(y1).round(2)) # [-1.61 -2.3 -1.61 -0.69]
print(np.log(y2).round(2)) # [-2.3 -2.3 -2.3 -0.36]

print(-t * np.log(y1).round(2)) # [-0. -0. -0. 0.69]
print(-t * np.log(y2).round(2)) # [-0. -0. -0. 0.36]

print(np.sum(-t * np.log(y1)).round(2)) # 0.69
print(np.sum(-t * np.log(y2)).round(2)) # 0.36

cee1 = np.sum(-np.log(0.5)).round(2)
cee2 = np.sum(-np.log(0.7)).round(2)

print(cee1) # 0.69
print(cee2) # 0.36

print(-np.log(0.5).round(2)) # 0.69
print(-np.log(0.7).round(2)) # 0.36
```

5. 4개의 입력 데이터에 대한 t(정답, label)와 a3(예측 점수, score, logit)가 다음과 같을 때
y(예측 확률), p(예측 인덱스), accuracy_cnt(예측 정확도)를 구하는 코드를 완성하고 결과를 정리하시오.
(N = 4, 클래스 개수 = 3 : 정수형 label = 0 / 1 / 2)

```
import numpy as np

def softmax(x):
    x = x - np.max(x, axis=1).reshape(-1,1)
    return np.exp(x) / np.sum(np.exp(x), axis=1).reshape(-1,1)

t = np.array([1, 0, 2, 1])
a3 = np.array([[0.1, 0.7, 0.1], [0.6, 0.2, 0.3], [0.8, 0.2, 0.1], [0.2, 0.6, 0.3]])
y = softmax(a3)          # 예측 확률
p = np.argmax(y, axis=1)  # 예측 인덱스
print(a3)
print(y.round(3))
print(p)
print(t)

accuracy_cnt = 0          # 정확하게 예측한 개수
for i in range(len(y)):
    if p[i] == t[i]: accuracy_cnt += 1

print(f"Accuracy = {accuracy_cnt / len(y)}") # 예측 정확도
```

a3 (예측 점수)	[[0.1 0.7 0.1] [0.6 0.2 0.3] [0.8 0.2 0.1] [0.2 0.6 0.3]]
y (예측 확률)	[[0.262 0.477 0.262] [0.415 0.278 0.307] [0.489 0.268 0.243] [0.278 0.415 0.307]]
p (예측 레이블)	[1 0 0 1]
t (실제 레이블)	[1 0 2 1]
Accuracy (예측 정확도)	Accuracy = 0.75

6. 입력 x에 드롭아웃 비율 0.3을 적용하는 아래 코드를 완성하고 결과를 정리하시오.

```
import numpy as np
np.random.seed(42)

x = np.array([[1, 2, 1, 3, 2], [2, 5, 3, 4, 1]])
dropout_ratio = 0.3
p = np.random.rand(*x.shape)
mask = p > dropout_ratio
x_drop = x * mask

print(x)
print(p.round(3))
print(mask)
print(x_drop)
```

(결과)

```
x
[[1 2 1 3 2] [2 5 3 4 1]]

p
[[0.375 0.951 0.732 0.599 0.156] [0.156  
0.058 0.866 0.601 0.708]]

mask
[[ True True True True False] [False False  
 True True True]]

x_drop
[[1 2 1 3 0] [0 0 3 4 1]]
```